

1 Qu'est-ce que le Développement Front-End ?

1.1 Définition

Développement Front-End

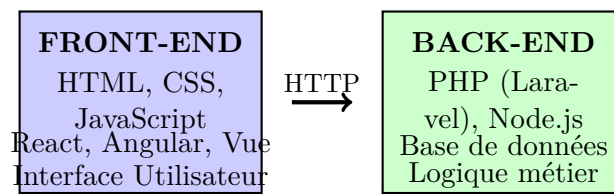
Le **développement front-end** consiste à créer la partie **visible et interactive** d'une application web ou mobile avec laquelle l'utilisateur interagit directement.

En d'autres termes : Tout ce que vous voyez et avec quoi vous interagissez dans votre navigateur = Front-End

1.2 Les 3 Piliers du Front-End

Technologie	Rôle
HTML	Structure et contenu de la page
CSS	Apparence et mise en forme (couleurs, tailles, positions)
JavaScript	Interactivité et logique (clics, animations, données dynamiques)

1.3 Front-End vs Back-End



Exemple concret :

- **Front-End :** Formulaire de connexion (champs, bouton, couleurs)
- **Back-End :** Vérifier si l'email et le mot de passe sont corrects dans la base de données

1.4 L'Évolution du Front-End

Époque	Approche	Exemple
2000-2010	Pages statiques HTML/CSS	Sites vitrines simples
2010-2015	jQuery	Manipulation DOM simplifiée
2015-2025	Frameworks modernes	React, Angular, Vue

2 Pourquoi Utiliser React ?

2.1 Le Problème avec JavaScript Vanilla

Imaginons qu'on veuille créer une application TODO list simple.

Avec JavaScript Vanilla, on doit :

1. Sélectionner les éléments DOM manuellement
2. Créer chaque élément HTML avec JavaScript
3. Gérer manuellement l'ajout/suppression d'éléments
4. Synchroniser manuellement les données et l'interface
5. Tout recréer quand les données changent

Exemple simple :

```
1 // Ajouter une tâche en JavaScript vanilla
2 function ajouterTache(texte) {
3   const li = document.createElement('li');
4   li.textContent = texte;
5
6   const boutonSupprimer = document.createElement('button');
7   boutonSupprimer.textContent = 'X';
8   boutonSupprimer.onclick = function() {
9     li.remove();
10  };
11
12  li.appendChild(boutonSupprimer);
13  document.getElementById('liste').appendChild(li);
14 }
```

Problèmes :

- Code répétitif et verbeux
- Difficile à maintenir quand l'application grandit
- Gestion manuelle du DOM = risque de bugs
- Pas de structure claire

2.2 La Solution : React

Qu'est-ce que React ?

React est une **bibliothèque JavaScript** créée par **Facebook (Meta)** en 2013 pour construire des interfaces utilisateur (SPA Applications) modernes de manière **simple, rapide et organisée**.

Même TODO avec React :

```
1 function Tache({ texte, onSupprimer }) {
2   return (
3     <li>
4       {texte}
5       <button onClick={onSupprimer}>X</button>
6     </li>
7   );
8 }
```

Avantages visibles :

- Code plus court et lisible
- Structure claire (composant)
- Réutilisable facilement
- React gère automatiquement le DOM

2.3 Les 5 Raisons d'Utiliser React

1. **Composants Réutilisables**
 - Créer des "briques" de code réutilisables
 - Un bouton, une carte, un formulaire = 1 composant
 - Réutiliser partout dans l'application
2. **DOM Virtuel (Performance)**
 - React ne met à jour que ce qui a changé
 - Pas besoin de recharger toute la page
 - Application très rapide
3. **JSX - HTML dans JavaScript**
 - Écrire du HTML directement dans JavaScript
 - Plus lisible et intuitif
 - Moins de code boilerplate
4. **Écosystème Énorme**
 - Des milliers de bibliothèques compatibles
 - Communauté très active
 - Beaucoup de ressources d'apprentissage
5. **Demandé par le Marché**
 - React est le framework front-end le plus utilisé
 - Énormément d'offres d'emploi
 - Utilisé par : Facebook, Instagram, Netflix, Airbnb, etc.

2.4 React vs Autres Frameworks

Framework	Avantages	Inconvénients	Popularité
React	Flexible, grande communauté, JSX	Nécessite d'autres bibliothèques	
Angular	Complet, TypeScript	Complexe, lourd	
Vue.js	Simple, léger	Moins d'emplois	

3 Qu'est-ce qu'une SPA (Single Page Application) ?

3.1 Définition

Single Page Application (SPA)

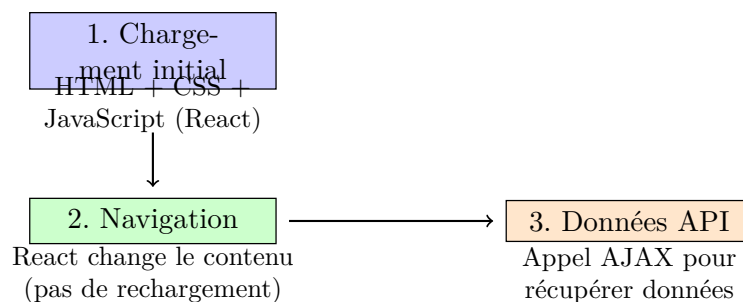
Une **SPA** est une application web qui charge **une seule page HTML** et met à jour dynamiquement le contenu sans recharger toute la page.

Exemple : Gmail, Facebook, Twitter - vous naviguez sans jamais voir la page se recharger complètement.

3.2 SPA vs MPA (Multi-Page Application)

MPA Traditionnelle	SPA Moderne
Chaque clic = nouvelle page chargée	Une seule page, contenu changé dynamiquement
Rechargement complet du navigateur	Pas de rechargement
page1.html, page2.html, page3.html	index.html + React
Lent (attente serveur)	Rapide (comme une app mobile)
Exemples : Sites vitrines classiques	Exemples : Gmail, Netflix, Trello

3.3 Comment Fonctionne une SPA ?



3.4 Avantages d'une SPA

- **Expérience utilisateur fluide** (comme une application mobile)
- **Rapidité** (pas de rechargement de page)
- **Réactivité** (mise à jour instantanée)
- **Séparation Front-End/Back-End** (API REST)

3.5 Inconvénients d'une SPA

- **Premier chargement plus long** (tout le JavaScript)
- **SEO plus complexe** (Google doit exécuter JS)
- **Nécessite JavaScript activé** (obligatoire)